

DVC ve lakeFS Yerel MinIO Performans Karşılaştırması

KONTROLLÜ
BENCHMARK
RAPORU

İki veri sürümünün yazılması, eski sürüme dönülmesi ve verinin tekrar yerel diske alınması karşılaştırılmıştır.

10.002 GiB	%35.8	0.230 sn
Her veri sürümünün boyutu	lakeFS sürüm yazma süresi avantajı	lakeFS zero-copy branch + stat

Yönetici özeti

Aynı Windows bilgisayarda, aynı yerel MinIO üzerinde ve aynı yapıya sahip iki 10 GB Parquet sürümüyle çalışıldı. Tek başarılı koşuda lakeFS, sürüm yazma ile materialized geri dönüşü birleştiren akışı DVC'den %38.1 daha kısa sürede tamamladı. Sonuçlar görelî karşılaştırma içindir; production sürelerini doğrudan tahmin etmez.

Benchmark koşulları: 21 Ağustos 2026 | Rapor: 24 Ağustos 2026

1. Amaç

Bu çalışma, aynı büyük Parquet veri kümesinin iki sürümünü DVC ve lakeFS ile versiyonlama ve eski sürüme geri dönme sürelerini karşılaştırmak için yapılmıştır.

Benchmark aşağıdaki sorulara odaklanır:

- V1 ve V2 sürümlerini MinIO tabanlı depolamaya yazmak ne kadar sürüyor?
- V1 referansına geri dönmek ne kadar sürüyor?
- V1 verisini yeniden yerel diskte kullanılabilir hâle getirmek ne kadar sürüyor?
- lakeFS'in zero-copy branch yaklaşımı ile DVC'nin Git pointer ve cache yaklaşımı pratikte nasıl ayrışıyor?

Bu test bir production performans testi değildir. Her iki araç da aynı bilgisayarda ve aynı yerel MinIO üzerinde çalıştırılarak kontrollü bir karşılaştırma yapılmıştır.

2. Çalışma ortamı

Özellik	Değer
İşletim sistemi	Windows 11 (10.0.26200)
Python	3.12.10
DVC	3.67.1
lakeFS istemcisi (lakectl)	1.79.0
MinIO	Yerel, http://127.0.0.1:9000
lakeFS API	Yerel, http://127.0.0.1:8002
DVC bucket	dvc-benchmark
lakeFS bucket	lakefs-benchmark
Koşu sayısı	Her araç için bir başarılı koşu

MinIO sürümü ve donanım ayrıntıları benchmark loguna kaydedilmemiştir. Bu nedenle sonuçlar farklı makinelerle mutlak performans karşılaştırması yapmak için kullanılmamalıdır.

3. Test verisi

V1 ve V2 aynı fiziksel yapıya, farklı rastgele tohumlara ve farklı içeriğe sahiptir.

Özellik	V1	V2
Dosya boyutu	10.002 GiB	10.002 GiB
Byte	10,739,950,326	10,739,950,326
Satır	83,886,080	83,886,080
Sütun	16	16
Row group	80	80
Veri tipi	16 adet float64 sütun	16 adet float64 sütun
Sıkıştırma	Yok	Yok
Rastgele tohum	20260821	20260822

Dosyalar aynı şema ve boyuta sahip olduğu için sürümler arasında yapısal olarak karşılaştırılabilir bir test oluşturulmuştur.

4. DVC benchmark akışı

DVC testi, ana projeden ayrı geçici bir Git ve DVC reposunda çalıştırılmıştır. DVC remote olarak yerel MinIO kullanılmıştır.

Akış:

1. V1 geçici çalışma alanındaki takip edilen dosya yoluna kopyalandı.
2. **dvc add** ile V1 hash'lendi, DVC cache'e alındı ve **.dvc** pointer oluşturuldu.
3. **dvc push** ile V1 MinIO'ya gönderildi.
4. V1 **.dvc** pointer'ı Git commitine kaydedildi.
5. Aynı çalışma alanı dosyası V2 ile değiştirildi.
6. V2 için tekrar **dvc add**, **dvc push** ve Git commit işlemleri yapıldı.
7. Git üzerinden V1 commitindeki **.dvc** pointer geri getirildi.
8. Cache mevcutken **dvc checkout** ile sıcak geri dönüş ölçüldü.
9. DVC cache boşaltılıp çalışma alanındaki dosya kaldırıldı.
10. **dvc pull** ile V1 MinIO'dan tekrar indirilerek soğuk geri dönüş ölçüldü.
11. Geri getirilen Parquet metadata'sındaki tohum kontrol edilerek dosyanın gerçekten V1 olduğu doğrulandı.

DVC adım süreleri

Adım	Süre	Throughput	Açıklama
V1'i çalışma alanına kopyalama	261.858 sn	39.114 MiB/sn	Benchmark staging işlemi
dvc add V1	536.887 sn	19.077 MiB/sn	Hash ve yerel cache oluşturma
dvc push V1	442.957 sn	23.123 MiB/sn	MinIO'ya gönderme
Git commit V1	0.440 sn	—	Pointer commit
V2'yi çalışma alanına kopyalama	296.200 sn	34.579 MiB/sn	Benchmark staging işlemi
dvc add V2	666.511 sn	15.367 MiB/sn	Hash ve yerel cache oluşturma
dvc push V2	457.256 sn	22.400 MiB/sn	MinIO'ya gönderme
Git commit V2	0.665 sn	—	Pointer commit
V1 pointer'ını Git'ten geri alma	0.162 sn	—	Yalnızca .dvc referansı değişir
Sıcak dvc checkout	283.526 sn	36.125 MiB/sn	V1 yerel cache'den materialize edilir
Soğuk dvc pull	853.461 sn	12.001 MiB/sn	V1 MinIO'dan boş cache'e çekilir ve materialize edilir

DVC toplamaları

Toplam	Süre
İki sürümü add + push + commit etme	2,104.715 sn (35 dk 4.7 sn)
Araç akışı + sıcak geri dönüş	2,388.403 sn (39 dk 48.4 sn)
Araç akışı + soğuk geri dönüş	2,958.338 sn (49 dk 18.3 sn)
Staging dahil uçtan uca soğuk akış	3,516.396 sn (58 dk 36.4 sn)
Ölçülen toplam duvar saati	3,812.840 sn (63 dk 32.8 sn)

git restore işleminin hızlı olması verinin geri geldiği anlamına gelmez. Bu adım yalnızca V1 hash'ini taşıyan **.dvc** pointer'ını seçer. Dosyanın çalışma alanına gelmesi için cache varsa **dvc checkout**, cache yoksa **dvc pull** gerekir.

5. lakeFS benchmark akışı

lakeFS sunucusu yerelde çalıştırılmış ve block storage olarak aynı yerel MinIO kullanılmıştır. V1 ve V2 aynı repository, **main** branch ve object path üzerinde ardışık iki commit olarak yazılmıştır.

Akış:

1. MinIO üzerinde ayrı storage namespace kullanan lakeFS repository oluşturuldu.
2. V1, **main/data/random_numbers.parquet** yoluna upload edildi.
3. V1, sürüm ve tohum metadata'sı ile lakeFS commitine kaydedildi.
4. Aynı object path V2 ile güncellendi.
5. V2 ayrı bir lakeFS commitine kaydedildi.
6. V1 commitinden **restore-v1** branch'i oluşturuldu.
7. Branch üzerindeki object metadata'sı okunarak V1 referansı doğrulandı.
8. Yerel dosya gereken senaryo için V1 branch'inden Parquet dosyası indirildi.
9. İndirilen dosyanın metadata tohumu kontrol edilerek V1 olduğu doğrulandı.

restore-v1 branch'inin oluşturulması zero-copy işlemdir. 10 GB veri kopyalanmaz; lakeFS yalnızca yeni branch referansını V1 commitine bağlar.

lakeFS adım süreleri

Adım	Süre	Throughput	Açıklama
Repository oluşturma	0.283 sn	—	Benchmark kurulumu
V1 upload	511.703 sn	20.016 MiB/sn	MinIO block storage'a yazma
V1 commit	0.851 sn	—	lakeFS data commit
V2 upload	837.122 sn	12.235 MiB/sn	Aynı object path'i V2 ile güncelleme
V2 commit	1.349 sn	—	lakeFS data commit
V1'den restore branch oluşturma	0.114 sn	—	Zero-copy branch
Restore edilen object için stat	0.115 sn	—	Referans ve metadata doğrulama
V1'i yerel diske indirme	478.612 sn	21.400 MiB/sn	Materialized read
İndirilen V1'i doğrulama	0.939 sn	—	Parquet metadata kontrolü

lakeFS toplamaları

Toplam	Süre
İki sürümü upload + commit etme	1,351.025 sn (22 dk 31.0 sn)
Zero-copy geri dönüş (branch + stat)	0.230 sn
Yerel materialized geri dönüş (branch + stat + download)	478.727 sn (7 dk 58.7 sn)
Sürüm yazma + zero-copy geri dönüş	1,351.254 sn (22 dk 31.3 sn)
Sürüm yazma + yerel materialized geri dönüş	1,829.751 sn (30 dk 29.8 sn)
Ölçülen toplam duvar saati	1,831.469 sn (30 dk 31.5 sn)

lakeFS tüketicisi veriyi doğrudan lakeFS URI üzerinden okuyabiliyorsa yerel indirme zorunlu değildir. Yerel bir dosyaya ihtiyaç varsa zero-copy branch işleminden sonra yine 10 GB verinin indirilme maliyeti oluşur.

6. Doğrudan karşılaştırma

Senaryo	DVC	lakeFS	Bu koşudaki fark
İki sürümü yazma ve commit etme	2,104.715 sn	1,351.025 sn	lakeFS %35.8 daha az süre kullandı
Eski sürüm referansını oluşturma/seçme	0.162 sn	0.114 sn	İkisi de saniyenin altında
Eski sürümü uzaktan yerel diske getirme	853.461 sn	478.612 sn	lakeFS %43.9 daha az süre kullandı
Sürüm yazma + soğuk/materialized geri dönüş	2,958.338 sn	1,829.751 sn	lakeFS %38.1 daha az süre kullandı
Ölçülen toplam duvar saati	3,812.840 sn	1,831.469 sn	Doğrudan eşdeğer olmayan yardımcı işlemleri de içerir

Sonuçların yorumu

- Sürüm yazma:** DVC iki aşamalı bir veri yolu kullanır. **dvc add** sırasında dosyayı hash'leyip yerel cache oluşturur, ardından **dvc push** ile cache içeriğini MinIO'ya gönderir. lakeFS benchmarkında kaynak dosya doğrudan object storage'a upload edilmiştir. Bu koşuda lakeFS sürüm yazma akışı yaklaşık **1.56x** daha kısa sürmüştür.
- Referans seviyesinde geri dönüş:** Her iki aracın metadata işlemi de saniyenin altındadır. DVC'de Git'ten eski **.dvc** pointer'ını seçmek 0.162 saniye, lakeFS'te V1 commitinden branch oluşturmak 0.114 saniye sürmüştür. Ancak semantikleri aynı değildir: DVC pointer değişiminden sonra yerel dosya için checkout/pull gerekir; lakeFS branch'i object-storage üzerinden doğrudan tüketilebilir.
- Yerel materialization:** Boş DVC cache ile V1'i MinIO'dan geri getirmek 853.461 saniye, lakeFS branch'inden doğrudan yerel hedefe indirmek 478.612 saniye sürmüştür. Bu koşuda lakeFS indirmesi yaklaşık **1.78x** daha kısa sürmüştür. DVC pull işlemi cache'i ve çalışma alanını hazırlarken lakeFS download tek hedef dosyaya yazdığı için bu sonuç saf ağ throughput karşılaştırması değildir; gerçek araç kullanım akışlarının karşılaştırmasıdır.
- Zero-copy etkisi:** lakeFS branch'ini kullanabilen Spark, Dagster veya başka bir uzak consumer için 0.230 saniyelik **branch + stat** sonrasında veri kullanılabilir durumdadır. Yerel dosya istemeyen bu senaryonun DVC benchmarkında doğrudan eşdeğeri yoktur; DVC'nin tipik tüketim akışı veriyi çalışma alanına materialize etmektir.

7. Geçerlilik ve sınırlamalar

- İki araç aynı bilgisayar ve aynı yerel MinIO üzerinde çalıştırıldığı için görelî karşılaştırma için ortak bir zemin sağlanmıştır.
- Yerel MinIO gerçek bir uzak object storage'ın ağ gecikmesini, bant genişliğini, TLS maliyetini ve servis tarafı ölçeklenmesini yansıtmaz.
- Kaynak dosyalar, DVC cache, lakeFS block storage ve indirilen çıktılar aynı fiziksel disk üzerinde bulunmuş olabilir. Bu durum disk I/O rekabeti oluşturabilir.
- Her araç için yalnızca bir başarılı koşu vardır. Ortalama, standart sapma veya güven aralığı hesaplanmamıştır.
- İşletim sistemi dosya cache'inin etkisi tamamen kontrol edilmemiştir.
- DVC warm checkout ölçülmüş, lakeFS için eşdeğer bir istemci cache senaryosu ölçülmemiştir.
- DVC staging kopyaları ayrıca ölçülmüştür; lakeFS kaynak dosyayı doğrudan upload ettiği için aynı staging adımı yoktur. Bu nedenle staging dahil toplam süre ana karşılaştırma metriği değildir.
- Paralel developer sayısı, eşzamanlı consumer'lar, branch merge süreleri, storage kullanımı ve production yetkilendirme maliyeti bu hız testinin kapsamında değildir.

Sonuç olarak bu çalışma production sürelerini tahmin etmekten çok, aynı yerel koşullar altında iki aracın gerçek kullanım akışlarındaki görelî maliyeti gösterir. Bu tek koşulda lakeFS hem sürüm yazma hem soğuk materialization akışını daha kısa sürede tamamlamıştır. DVC ise Git commitleriyle doğal bütünleşme karşılığında hash/cache oluşturma ve pointer yönetimi adımlarını taşımaktadır.

8. Kaynak dosyalar

- DVC benchmark scripti: [`scripts/benchmark\\_dvc\\_minio.py`](#)
- lakeFS benchmark scripti: [`scripts/benchmark\\_lakefs\\_minio.py`](#)
- Veri üretme scripti: [`scripts/generate\\_benchmark\\_data.py`](#)
- Başarılı DVC JSON kaydı: [`dvc\\_minio\\_benchmark\\_20260821T091735Z\\_44a24ac8.json`](#)
- Başarılı lakeFS JSON kaydı: [`lakefs\\_minio\\_benchmark\\_20260821T110551Z\\_5a30af20.json`](#)

9. Mimari değerlendirme: partitioning, cache ve worker ömrü

Hazır olmak ne demek?

lakeFS branch oluşturulduğunda veri compute belleğinde veya yerel diskte hazır değildir; ilgili sürüm doğrudan okunmaya hazırdır. İşlenecek byte'lar yine MinIO'dan worker'lara taşınır. Zero-copy branching veri transferini değil, sürüm için önceden fiziksel kopya oluşturma ihtiyacını kaldırır.

Veri transferi açısından gerçek fark

DVC	lakeFS
MinIO -> dvc pull -> yerel cache/workspace -> processing	lakeFS branch -> processing sırasında gerekli object/Parquet parçalarını MinIO'dan okuma

Tek bir job bütün datasetin tamamını tarıyorsa iki yöntemde de yaklaşık aynı miktarda veri compute ortamına taşınır. lakeFS'in hız avantajı, processing öncesinde zorunlu DVC cache oluşturma ve workspace materialization adımlarının bulunmamasıdır. Buna karşılık aynı veri birçok kez okunacaksa kalıcı DVC cache'i daha hızlı olabilir.

DVC'de partitioning ile kazanım

Dataset tarih/uçuş gibi küçük Parquet partition'larına ayrılırsa Dagster yalnızca ilgili batch yolunu dvc pull ile getirebilir. Böylece DVC, veri transfer miktarı bakımından lakeFS'e yaklaşır. Ancak tek bir 10 GB Parquet dosyası hedeflenirse DVC pull dosyanın tamamını cache'e alır; Spark + lakeFS ise işlem sırasında gereken kolonları, partition'ları ve uygun row group'ları remote storage'dan okuyabilir.

Kısa ömürlü worker etkisi

Kubernetes, ECS veya benzeri ortamlarda processing worker'ları her job için açılıp kapanıyorsa kalıcı DVC cache çoğu zaman kullanılamaz. Her yeni worker'ın aynı veriyi tekrar pull etmesi ve cache hazırlaması gerekir. lakeFS bu durumda branch URI'sini doğrudan input yaparak processing'i ön indirme olmadan başlatır; gereken byte'lar job sırasında MinIO'dan okunur.

Karar tablosu

Çalışma koşulu	Daha uygun yaklaşım	Gerekçe
Uzun ömürlü worker ve kalıcı cache	DVC	Aynı veri tekrar okunurken remote transfer tekrarlanmaz.
Tarih/uçuş bazlı partition ve hedefli pull	DVC rekabetçi	Yalnızca gereken partition indirilerek toplam I/O azaltılır.
Kısa ömürlü Kubernetes/ECS worker	lakeFS	Her job'da DVC cache ve workspace yeniden hazırlanmadan doğrudan remote okuma başlar.
Çok sayıda uzak consumer ve merkezi veri portalı	lakeFS	Aynı branch/tag URI'si merkezi ve doğrudan okunabilen bir dataset görünümü sağlar.
Aynı dataset üzerinde tekrarlı tam taramalar	DVC avantaj kazanabilir	Kalıcı yerel cache, sonraki okumaları remote erişimden kurtarabilir.

Proje için değerlendirme

Merkezi veri portalı, birden fazla uzak consumer ve kısa ömürlü processing worker'ları hedefleniyorsa lakeFS daha doğal bir veri katmanıdır. Worker cache'lerinin korunabildiği, datasetin iyi partition edildiği ve Git merkezli lineer bir akışın yeterli olduğu durumda DVC daha düşük operasyon maliyetiyle benzer veri-transfer verimliliğine yaklaşabilir.

Teknik kaynaklar: DVC pull - <https://dvc.org/doc/command-reference/pull> | lakeFS Spark entegrasyonu - <https://docs.lakefs.io/integrations/spark.html>